PART 1: Source Code - surge_predictor.py

```
"""
Real-Time Dynamic Pricing & Demand Surge Predictor
Ridesharing / Food Delivery

Models used (4): Logistic Regression, KNN, SVM, Random Forest
(CNN is skipped on purpose: our data is tabular, CNNs are meant for images/sequences.)

Run:  pip install numpy pandas scikit-learn matplotlib joblib
      python surge_predictor.py
"""
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import joblib

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, f1_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

RNG = np.random.default_rng(42)

# ---------------------------------------------------------------------
# 1. DATA (synthetic, realistic). Replace with real data if you have it.
# ---------------------------------------------------------------------
def generate_data(n=8000):
    hour = RNG.integers(0, 24, n)
    day = RNG.integers(0, 7, n)                      # 0=Mon ... 6=Sun
    is_weekend = (day >= 5).astype(int)
    rush = (((hour >= 8) & (hour <= 10)) | ((hour >= 17) & (hour <= 20))).astype(int)
    late_night = ((hour >= 22) | (hour <= 3)).astype(int)
    rain_mm = np.where(RNG.random(n) < 0.25, RNG.gamma(2, 3, n), 0.0)
    temp_c = RNG.normal(28, 6, n)
    event_nearby = (RNG.random(n) < 0.08).astype(int)    # concert / match ending
    traffic_index = np.clip(RNG.normal(4 + 3 * rush + 1.5 * rain_mm / 10, 1.5, n), 0, 10)

    # demand (ride requests in a zone per 10 min) and supply (idle drivers)
    demand = (30 + 25 * rush + 15 * is_weekend * late_night + 2.2 * rain_mm
              + 45 * event_nearby + RNG.normal(0, 6, n)).clip(5)
    supply = (40 - 6 * rush - 0.4 * rain_mm - 5 * late_night
              - 4 * event_nearby + RNG.normal(0, 5, n)).clip(3)
    ratio = demand / supply
    avg_wait_min = (3 + 2.5 * ratio + RNG.normal(0, 1, n)).clip(1)

    df = pd.DataFrame(dict(hour=hour, day_of_week=day, is_weekend=is_weekend,
                           is_rush_hour=rush, rain_mm=rain_mm.round(1),
                           temp_c=temp_c.round(1), event_nearby=event_nearby,
                           traffic_index=traffic_index.round(1),
                           active_requests=demand.round(0),
                           available_drivers=supply.round(0),
                           avg_wait_min=avg_wait_min.round(1)))
    df["demand_supply_ratio"] = (df.active_requests / df.available_drivers).round(2)

    # Target: surge level (noisy so models are not trivially perfect)
    score = df.demand_supply_ratio + RNG.normal(0, 0.15, n)
    df["surge_level"] = pd.cut(score, [-np.inf, 1.1, 1.9, np.inf], labels=[0, 1, 2]).astype(int)
    return df

LABELS = {0: "No surge", 1: "Moderate surge", 2: "High surge"}
MULTIPLIER = {0: 1.0, 1: 1.5, 2: 2.2}     # business rule: class -> price multiplier

FEATURES = ["hour", "day_of_week", "is_weekend", "is_rush_hour", "rain_mm", "temp_c",
            "event_nearby", "traffic_index", "active_requests", "available_drivers",
            "avg_wait_min", "demand_supply_ratio"]

# ---------------------------------------------------------------------
# 2. MODELS (scaling matters for LR, KNN, SVM; RF does not need it)
# ---------------------------------------------------------------------
```

PART 1: Source Code - surge_predictor.py

```
def build_models():
    return {
        "Logistic Regression": Pipeline([("sc", StandardScaler()),
                                         ("m", LogisticRegression(max_iter=1000))]),
        "KNN": Pipeline([("sc", StandardScaler()),
                         ("m", KNeighborsClassifier(n_neighbors=7))]),
        "SVM": Pipeline([("sc", StandardScaler()),
                         ("m", SVC(kernel="rbf", C=3, probability=True))]),
        "Random Forest": RandomForestClassifier(n_estimators=250, max_depth=12,
                                                random_state=42, n_jobs=-1),
    }

# ---------------------------------------------------------------------
# 3. TRAIN + EVALUATE
# ---------------------------------------------------------------------
def main():
    df = generate_data()
    df.to_csv("surge_dataset.csv", index=False)
    print("Class distribution:\n", df.surge_level.map(LABELS).value_counts(), "\n")

    X, y = df[FEATURES], df["surge_level"]
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                              stratify=y, random_state=42)
    models, results, fitted = build_models(), [], {}

    for name, model in models.items():
        model.fit(X_tr, y_tr)
        pred = model.predict(X_te)
        cv = cross_val_score(model, X_tr, y_tr, cv=5, scoring="accuracy").mean()
        results.append(dict(Model=name,
                            Accuracy=accuracy_score(y_te, pred),
                            F1_macro=f1_score(y_te, pred, average="macro"),
                            CV_Accuracy=cv))
        fitted[name] = (model, pred)
        print(f"===== {name} =====")
        print(classification_report(y_te, pred, target_names=list(LABELS.values())))

    res = pd.DataFrame(results).sort_values("F1_macro", ascending=False).round(4)
    print("\nMODEL COMPARISON\n", res.to_string(index=False))
    res.to_csv("model_comparison.csv", index=False)

    # --- Plots ---
    fig, axes = plt.subplots(2, 2, figsize=(11, 9))
    for ax, (name, (_, pred)) in zip(axes.ravel(), fitted.items()):
        ConfusionMatrixDisplay(confusion_matrix(y_te, pred),
                               display_labels=["None", "Mod", "High"]).plot(ax=ax, colorbar=False)
        ax.set_title(name)
    plt.tight_layout(); plt.savefig("confusion_matrices.png", dpi=130); plt.close()

    rf = fitted["Random Forest"][0]
    imp = pd.Series(rf.feature_importances_, index=FEATURES).sort_values()
    imp.plot.barh(figsize=(7, 5), title="Random Forest – What drives surge?")
    plt.tight_layout(); plt.savefig("feature_importance.png", dpi=130); plt.close()

    # --- Save best model ---
    best_name = res.iloc[0]["Model"]
    joblib.dump(fitted[best_name][0], "best_surge_model.joblib")
    print(f"\nBest model: {best_name}  (saved to best_surge_model.joblib)")

    # ---------------------------------------------------------------
    # 4. LIVE DEMO: predict surge + dynamic price for a new situation
    # ---------------------------------------------------------------
    best = fitted[best_name][0]
    base_fare = 120.0   # INR
    scenarios = {
        "Normal afternoon": dict(hour=14, day_of_week=2, is_weekend=0, is_rush_hour=0,
                                 rain_mm=0, temp_c=30, event_nearby=0, traffic_index=3.5,
                                 active_requests=28, available_drivers=42, avg_wait_min=5),
        "Rush hour + rain": dict(hour=18, day_of_week=4, is_weekend=0, is_rush_hour=1,
                                 rain_mm=14, temp_c=24, event_nearby=0, traffic_index=8.5,
                                 active_requests=75, available_drivers=30, avg_wait_min=11),
        "Concert just ended": dict(hour=23, day_of_week=5, is_weekend=1, is_rush_hour=0,
                                   rain_mm=0, temp_c=26, event_nearby=1, traffic_index=7,
                                   active_requests=110, available_drivers=22, avg_wait_min=18),
    }
```

PART 1: Source Code - surge_predictor.py

```
    print("\nLIVE PRICING DEMO (base fare Rs.%.0f)" % base_fare)
    for title, s in scenarios.items():
        s["demand_supply_ratio"] = round(s["active_requests"] / s["available_drivers"], 2)
        row = pd.DataFrame([s])[FEATURES]
        cls = int(best.predict(row)[0])
        conf = best.predict_proba(row)[0].max()
        print(f"{title:22s} -> {LABELS[cls]:15s} x{MULTIPLIER[cls]}  "
              f"fare Rs.{base_fare * MULTIPLIER[cls]:.0f}  (confidence {conf:.0%})")


if __name__ == "__main__":
    main()
```

PART 2: Program Output (console)

```
Class distribution:
 surge_level
No surge          3768
Moderate surge    2784
High surge        1448
Name: count, dtype: int64

===== Logistic Regression =====
                precision    recall  f1-score   support

      No surge       0.92      0.93      0.93       754
Moderate surge       0.86      0.85      0.86       557
    High surge       0.92      0.91      0.91       289

      accuracy                           0.90      1600
     macro avg       0.90      0.90      0.90      1600
  weighted avg       0.90      0.90      0.90      1600

===== KNN =====
                precision    recall  f1-score   support

      No surge       0.89      0.94      0.92       754
Moderate surge       0.82      0.80      0.81       557
    High surge       0.90      0.82      0.86       289

      accuracy                           0.87      1600
     macro avg       0.87      0.85      0.86      1600
  weighted avg       0.87      0.87      0.87      1600

===== SVM =====
                precision    recall  f1-score   support

      No surge       0.92      0.94      0.93       754
Moderate surge       0.86      0.85      0.85       557
    High surge       0.91      0.88      0.89       289

      accuracy                           0.90      1600
     macro avg       0.90      0.89      0.89      1600
  weighted avg       0.90      0.90      0.90      1600

===== Random Forest =====
                precision    recall  f1-score   support

      No surge       0.92      0.93      0.93       754
Moderate surge       0.86      0.86      0.86       557
    High surge       0.93      0.89      0.91       289

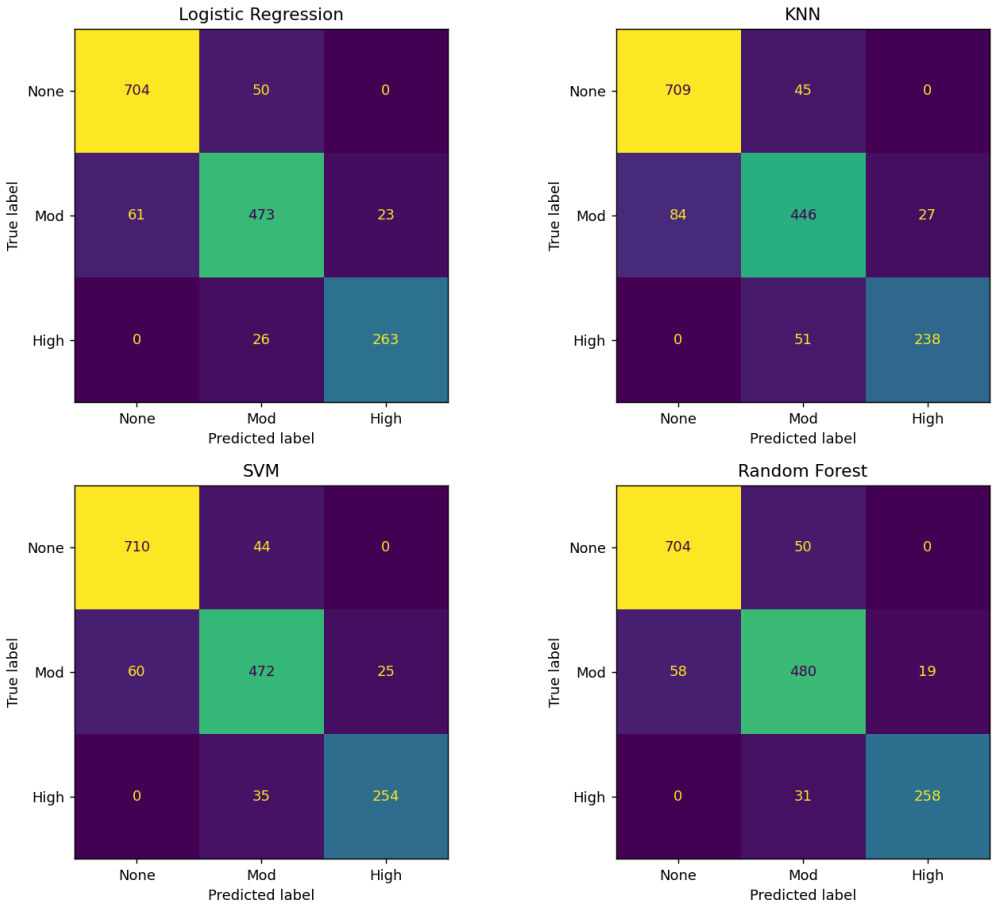
      accuracy                           0.90      1600
     macro avg       0.90      0.90      0.90      1600
  weighted avg       0.90      0.90      0.90      1600


MODEL COMPARISON
               Model  Accuracy  F1_macro  CV_Accuracy
      Random Forest    0.9012    0.8997       0.8922
Logistic Regression    0.9000    0.8990       0.8967
                SVM    0.8975    0.8927       0.8905
                KNN    0.8706    0.8625       0.8562

Best model: Random Forest  (saved to best_surge_model.joblib)

LIVE PRICING DEMO (base fare Rs.120)
Normal afternoon       -> No surge        x1.0  fare Rs.120  (confidence 100%)
Rush hour + rain       -> High surge      x2.2  fare Rs.264  (confidence 100%)
Concert just ended     -> High surge      x2.2  fare Rs.264  (confidence 100%)
```

PART 3: Output - Confusion Matrices



PART 3: Output - Feature Importance (Random Forest)

